

Universidad Tecnológica de Tehuacán

Programa Educativo
ING. EN DESARROLLO Y GESTIÓN DE SOFTWARE
ASIGNATURA:

Desarrollo Web Integral

TÍTULO DE TRABAJO:

Arquitectura del sistema

Nombre del Docente:

José Miguel Carrera Pacheco

NOMBRES DE LOS INTEGRANTES:

Jose Julian Alvarez Flores

Arturo Castañeda Serrano

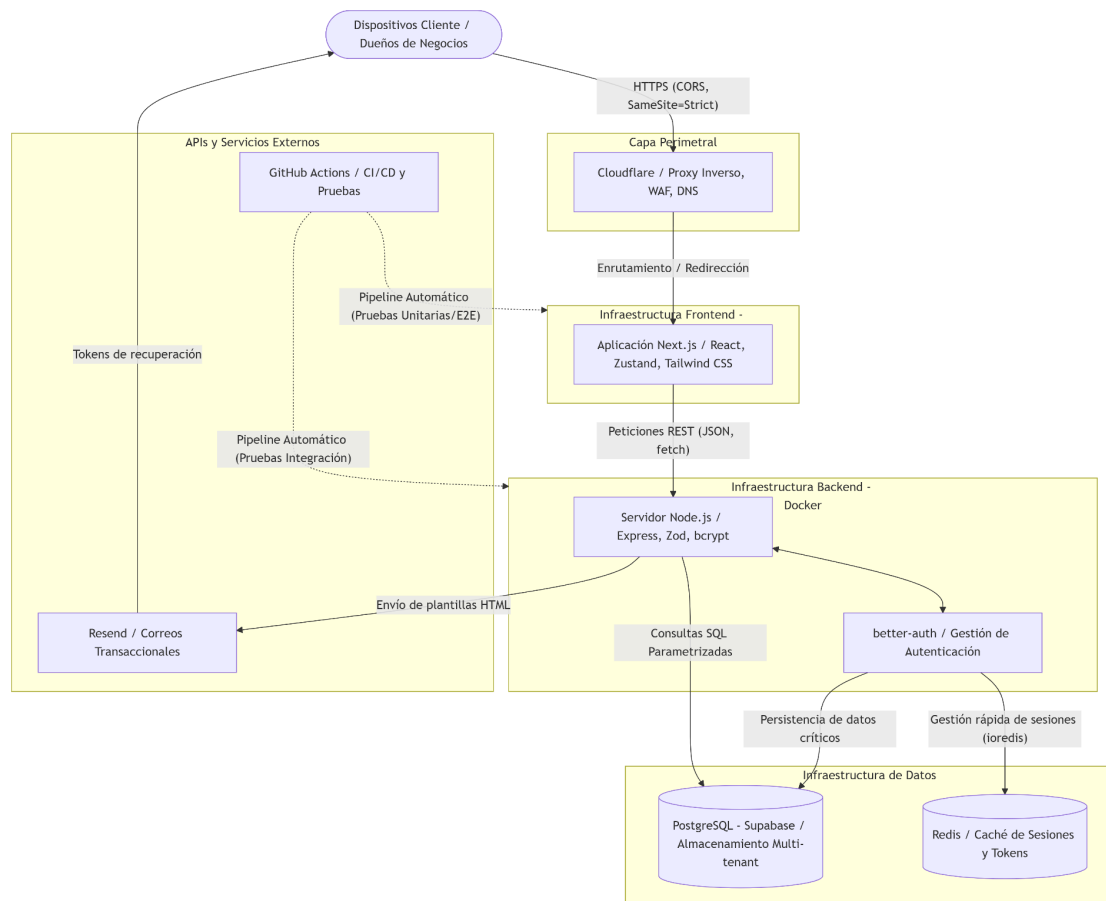
Katherine Daniela Gómez Merino

Armando Valerio Salmeron

Cuatrimestre: 9° Grupo: "B"

Diagrama de arquitectura

Diagrama de arquitectura.png



Modelo de datos

Tabla:Negocio

Campo	Tipo	Descripción
id_negocio	UUID PK	Identificador único del negocio o inquilino.
nombre	VARCHAR	Nombre comercial del negocio.
subdominio	VARCHAR	Subdominio asignado para su acceso en la plataforma.
activo	BOOLEAN	Indica si el negocio está habilitado en el sistema (por defecto true).

Tabla: user

Campo	Tipo	Descripción
id	VARCHAR PK	Identificador único del usuario.
id_negocio	UUID FK	Relación con la tabla negocio (A qué empresa pertenece).
role	VARCHAR	Nivel de acceso en el sistema (ej. 'admin').

id_usuario_creador	VARCHAR FK	Relación con <code>user</code> . Identifica qué administrador creó esta cuenta.
name	TEXT	Nombre completo del usuario.
email	TEXT UNIQUE	Correo electrónico, usado para el login.
emailVerified	BOOLEAN	Indica si el correo electrónico ha sido validado.
image	TEXT NULL	URL de la foto de perfil del usuario.
createdAt	TIMESTAMPTZ	Fecha y hora en la que se creó la cuenta.
updatedAt	TIMESTAMPTZ	Fecha y hora de la última modificación de la cuenta.

Tabla: account

Campo	Tipo	Descripción
id	TEXT PK	Identificador único del registro de autenticación.
accountId	TEXT	ID de la cuenta en el proveedor externo.

providerId	TEXT	Proveedor de identidad (ej. Google, Credentials).
userId	TEXT FK	Relación con la tabla user (A quién pertenecen estas credenciales).
accessToken	TEXT NULL	Token de acceso temporal.
refreshToken	TEXT NULL	Token para renovar la sesión.
password	TEXT NULL	Contraseña encriptada (si aplica login tradicional).
createdAt	TIMESTAMP TZ	Fecha de creación del registro.

Tabla: invitacion

Campo	Tipo	Descripción
id_invitacion	UUID PK	Identificador único de la invitación.
id_negocio	UUID FK	Negocio al que se está invitando a participar.
inviter_user_id	VARCHAR FK	ID del usuario (user) que envió la invitación.

email_invitado	VARCHAR	Correo al que se envía la solicitud de acceso.
role_asignado	VARCHAR	Rol que tendrá el usuario si acepta la invitación.
token_seguridad	VARCHAR	Código único para validar el enlace de invitación.
expiresat	TIMESTAMP	Fecha y hora en la que el enlace deja de ser válido.
aceptada	BOOLEAN	Indica si el usuario ya se registró mediante esta invitación.

Tabla: categoria

Campo	Tipo	Descripción
id_categoria	UUID PK	Identificador único de la categoría.
id_negocio	UUID FK	Negocio al que pertenece esta clasificación.
nombre	VARCHAR	Nombre de la categoría (ej. "Bebidas", "Lácteos").
descripcion	VARCHAR NULL	Detalles adicionales de la categoría.

activo	BOOLEAN	Indica si la categoría está disponible para usarse.
--------	---------	---

Tabla: producto

Campo	Tipo	Descripción
id_producto	UUID PK	Identificador único del producto.
id_negocio	UUID FK	Negocio al que pertenece el catálogo.
id_categoria	UUID FK	Relación con la tabla categoria .
codigo_barras	VARCHAR NULL	Código de barras para escaneo en punto de venta.
nombre	VARCHAR	Nombre comercial del producto.
precio_actual	DOUBLE	Precio de venta al público.
stock_minimo_sugerido	INTEGER	Cantidad mínima ideal antes de requerir reabastecimiento.
activo	BOOLEAN	Indica si el producto se sigue comercializando.

Tabla: lote_inventario

Campo	Tipo	Descripción
id_lote	UUID PK	Identificador único del lote ingresado.
id_producto	UUID FK	Relación con producto (A qué artículo pertenece este lote).
codigo_lote	VARCHAR	Código impreso por el fabricante para rastreo.
fecha_ingreso	DATE	Día en que el lote entró al almacén.
fecha_caducidad	DATE NULL	Fecha límite de consumo (crucial para perecederos).
cantidad_inicial	INTEGER	Número de piezas con las que entró este lote.
activo	BOOLEAN	Indica si aún quedan unidades disponibles o vigentes.

Tabla: venta

Campo	Tipo	Descripción
id_venta	UUID PK	Identificador único del recibo o ticket de venta.

id_negocio	UUID FK	Sucursal donde se realizó la operación.
userid	VARCHAR FK	Usuario o cajero que procesó la venta.
fecha_transaccion	TIMESTAMP	Fecha y hora exacta del pago.

Tabla: detalle_va_venta

Campo	Tipo	Descripción
id_detalle	UUID PK	Identificador único del movimiento.
id_venta	UUID FK	Relación con la tabla venta (A qué ticket pertenece).
id_lote	UUID FK	Crucial: Relación con lote_inventario . Se descuenta de un lote específico, no del producto general.
cantidad_sold	INTEGER	Número de unidades vendidas de este lote.
precio_unitario	DOUBLE	Precio al que se vendió en ese momento específico.

Tabla: tipo_alerta y alerta

Campo	Tipo	Descripción
-------	------	-------------

id_tipo_alerta	UUID PK	(<i>Tabla tipo_alerta</i>) Identificador de la clasificación del aviso (ej. "Caducidad cercana").
id_alerta	UUID PK	(<i>Tabla alerta</i>) Identificador único del evento generado.
id_producto	UUID FK	Producto que detonó el aviso de precaución.
fecha_emision	DATE	Día en que el sistema detectó el problema y generó el aviso.
resuelta	BOOLEAN	Indica si un usuario ya atendió y solucionó el problema advertido.

Tabla: interests

Campo	Tipo	Descripción
id	BIGINT PK	Identificador único autoincrementable.
nombre	VARCHAR	Nombre de la persona interesada en el software.
negocio	VARCHAR	Nombre de la empresa del prospecto.
telefono	VARCHAR	Número de contacto.
created_at	TIMESTAMPTZ	Fecha en la que llenaron el formulario.

Justificación del stack

La selección tecnológica para el desarrollo de este proyecto SaaS se ha diseñado buscando un equilibrio óptimo entre rendimiento, alta escalabilidad, seguridad robusta y una curva de aprendizaje adecuada para la experiencia previa del equipo. Para el ecosistema del frontend, se optó por una arquitectura monolítica utilizando React en conjunto con Next.js. Esta decisión se fundamenta en la capacidad de Next.js para facilitar la creación de interfaces altamente dinámicas y un enrutamiento intuitivo, apoyado por una amplia comunidad. El diseño visual se realizará con Tailwind CSS, lo que permite un desarrollo ágil y responsivo mediante clases utilitarias, manteniendo la consistencia en todas las vistas sin sobrecargar los estilos. Además, la gestión del estado global recae en Zustand, una herramienta ligera y directa que resulta indispensable para manejar en memoria los datos del negocio, como las alertas de caducidad y las sesiones de los usuarios, minimizando re-renderizados innecesarios y optimizando el rendimiento en los dispositivos de los clientes.

El núcleo de la lógica de negocio y el procesamiento de datos operará sobre un backend construido con Node.js y Express, tipado de manera estricta mediante TypeScript. La adopción de TypeScript es una decisión estratégica para la prevención de errores en tiempo de desarrollo y la creación de un código más predecible y auto-documentado. Al seguir un modelo de arquitectura por capas, el sistema garantiza un mantenimiento a largo plazo mucho más sencillo. Para asegurar que los datos procesados sean correctos desde el origen, se integra Zod, proporcionando una capa de validación estricta y sanitización de entradas que previene fallos internos antes de que interactúen con los servicios pesados, como el cálculo de reabastecimiento o el análisis de ventas.

Para la persistencia y gestión de datos, la arquitectura exige un modelo relacional robusto debido a la naturaleza de las transacciones de inventario, por lo que PostgreSQL alojado en Supabase fue la elección ideal. Supabase no solo ofrece un entorno optimizado para PostgreSQL, sino que aporta capacidades nativas de pooling de conexiones y respaldos automáticos, lo que reduce costos operativos y de infraestructura. Para manejar la concurrencia y optimizar la escalabilidad del modelo multi-tenant, se introduce Redis como almacenamiento en memoria. Redis asume la carga de gestionar las sesiones activas, la caché de tokens, liberando a la base de datos principal de operaciones repetitivas y permitiendo una respuesta ultrarrápida del servidor. Todo esto se integra de forma transparente con better-auth, un marco centralizado que gestiona la autenticación de los tenants mediante cookies seguras (HTTP-Only, SameSite=Strict), protegiendo al sistema de

vulnerabilidades de sesión y aplicando un estricto control de acceso basado en roles (RBAC).

Finalmente, la infraestructura y el ciclo de vida del software están diseñados para maximizar la seguridad, la disponibilidad y la automatización. El frontend se apoya en la CDN global de Vercel para una entrega de contenido casi instantánea, mientras que el backend corre aislado y de forma reproducible en contenedores de Docker. En la capa perimetral, Cloudflare actúa como un escudo vital, proporcionando mitigación de ataques DDoS y un firewall de aplicaciones web (WAF). Para garantizar la fiabilidad del código antes de llegar a producción, se diseñó un esquema integral de pruebas (unitarias con Jest, E2E con Cypress y de rendimiento con K6) automatizado enteramente a través de GitHub Actions. Esto asegura que cualquier nueva integración cumpla con los estándares de calidad y cobertura, manteniendo el sistema estable, seguro y siempre disponible para la gestión crítica de los inventarios de los clientes.

Flujo del sistema

1. Inicio de Sesión (Autenticación)

1. **Cliente - Entrada y Validación:** El usuario introduce su correo y contraseña en el formulario de login. El esquema `loginSchema` de Zod valida que el correo sea válido y la contraseña cumpla con los requisitos mínimos de longitud.
2. **Cliente - Solicitud:** Si la validación es correcta, el `AuthContext` realiza una petición `POST` al endpoint `/api/auth/sign-in/email` del servidor API.
3. **Servidor - Procesamiento de Autenticación:** El middleware de Better-Auth (`toNodeHandler`) intercepta la petición, verifica si el correo existe en la base de datos PostgreSQL y utiliza la función de verificación para comparar la contraseña ingresada con el hash guardado.
4. **Servidor - Sesión y Caché:** Tras la verificación exitosa, Better-Auth registra la sesión activa en el almacenamiento secundario de **Redis** para optimizar peticiones futuras y genera un token de sesión.
5. **Respuesta y Redirección:** El servidor devuelve un código `200 OK` con los datos del usuario y establece la cookie segura de sesión (`auth.session` con atributos `HttpOnly`, `Secure` y `SameSite=Strict`). El frontend actualiza el estado global de Zustand/AuthContext y redirige al usuario a `/dashboard`.

2. Registro de Productos

1. **Cliente - Entrada:** El usuario ingresa la información del producto (código de barras, nombre, precio actual, stock mínimo sugerido e id de categoría) en el formulario de la interfaz.

2. **Cliente - Solicitud:** El frontend realiza una petición `POST` a `/products` enviando los datos y adjuntando de forma automática las cookies de sesión.
3. **Servidor - Validación de Sesión y Datos:** El middleware `requireAuth` comprueba la validez de la sesión usando `auth.api.getSession`. Si es válida, asocia los datos del usuario (incluyendo el `id_negocio` del tenant) al objeto `req.user`. Posteriormente, el middleware `validateDataBody(createProductoSchema)` valida la estructura y tipo de los campos del body mediante Zod.
4. **Servidor - Base de Datos:** `createProductController` extrae los datos y llama a `createProductService` para ejecutar un `INSERT` en la tabla `public.producto` en Supabase, asociando el producto de forma unívoca al `id_negocio`.
5. **Respuesta y Actualización:** El servidor retorna un `201 Created` con los datos del producto creado. El cliente actualiza el estado de Zustand (añadiendo el producto al catálogo local) y muestra una notificación de éxito en la interfaz.

3. Edición de Productos

1. **Cliente - Entrada:** En el formulario de edición de producto, el usuario modifica los campos deseados (código de barras, nombre, precio, stock mínimo sugerido o categoría) y hace clic en guardar cambios.
2. **Cliente - Solicitud:** El cliente realiza una petición `PUT` a `/products/:id_producto` con la cookie de sesión activa.
3. **Servidor - Validación:** Primero, `requireAuth` valida la sesión del usuario y rescata su `id_negocio`. Luego, los validadores de Zod corroboran que el `id_producto` de los parámetros sea un UUID válido (`productoldParamSchema`) y que los campos del body cumplan con las reglas de cambio parcial (`updateProductoSchema`).
4. **Servidor - Base de Datos:** `updateProductController` llama a `updateProductService`, el cual ejecuta una consulta `UPDATE` sobre la tabla `public.producto` asegurándose de filtrar por el `id_producto` y por el `id_negocio` para evitar accesos cruzados multi-tenant.
5. **Respuesta y Actualización:** Retorna `200 OK` junto con el producto modificado. Zustand actualiza el producto en memoria y el catálogo del dashboard refleja los cambios en tiempo real.

4. Eliminación de Productos

1. **Cliente - Confirmación:** El usuario pulsa el botón de eliminar producto y confirma la acción en un cuadro de diálogo.
2. **Cliente - Solicitud:** Se envía una solicitud `DELETE` a `/products/:id_producto` al backend.
3. **Servidor - Validación:** `requireAuth` verifica la sesión y `validateDataParams(productoldParamSchema)` asegura que el parámetro sea un UUID válido.

4. **Servidor - Base de Datos:** El controlador invoca a `deleteProductService` para remover el registro en la base de datos (o marcarlo como inactivo), restringido rigurosamente al `id_negocio` del usuario autenticado.
5. **Respuesta y Actualización:** El servidor responde con un código `204 No Content` indicando el éxito. El frontend remueve el producto del estado global de Zustand para refrescar la vista.

5. Registro de Lotes

1. **Cliente - Entrada:** El usuario selecciona un producto y rellena el formulario del lote: código de lote, cantidad inicial, fecha de ingreso y fecha de caducidad.
2. **Cliente - Solicitud:** Se envía un `POST` a `/lote` incluyendo los datos ingresados y la cookie de autenticación.
3. **Servidor - Validación:** El middleware `requireAuth` valida los datos de sesión del usuario. A continuación, el middleware `validateDataBody(createLoteSchema)` comprueba los tipos de datos y ejecuta la validación (refinement) para asegurar que la fecha de caducidad no sea anterior a la fecha de ingreso.
4. **Servidor - Base de Datos:** El controlador ejecuta `createLoteService` que realiza el `INSERT` en la tabla `public.lote` en Supabase, registrando la cantidad inicial y asociándola al `id_producto`.
5. **Respuesta y Actualización:** Retorna `201 Created` con el lote registrado. El frontend actualiza los niveles de stock en el inventario global y notifica la operación exitosa.

6. Edición de Lotes

1. **Cliente - Entrada:** El usuario edita la cantidad inicial, el código del lote o las fechas desde el formulario de edición de lote.
2. **Cliente - Solicitud:** Se realiza una llamada `PUT` al endpoint `/lote/:id_lote`.
3. **Servidor - Validación:** `requireAuth` valida la sesión del usuario. Se verifica que el parámetro `id_lote` sea válido. Finalmente, Zod valida las reglas de modificación de lote (`updateLoteSchema`) y corrobora de nuevo la consistencia lógica de las fechas.
4. **Servidor - Base de Datos:** El servicio `updateLoteService` comprueba que el lote pertenezca a un producto asociado al negocio del tenant logueado y realiza el `UPDATE` correspondiente de los datos del lote.
5. **Respuesta y Actualización:** El servidor retorna un `200 OK` con los datos actualizados del lote. Zustand actualiza el inventario local.

7. Eliminación de Lotes

1. **Cliente - Confirmación:** El usuario confirma la eliminación de un lote específico desde la vista del inventario.
2. **Cliente - Solicitud:** El cliente envía una petición `DELETE` a `/lote/:id_lote`.

3. **Servidor - Validación:** `requireAuth` comprueba la autenticidad de la sesión y `loteldParamSchema` valida el ID del lote.
4. **Servidor - Base de Datos:** `deleteLoteController` ejecuta `deleteLoteService` que borra el lote de la base de datos Postgres tras verificar que el negocio es propietario legítimo del producto asociado a ese lote.
5. **Respuesta y Actualización:** Retorna `204 No Content`. El frontend remueve el lote de la lista local en Zustand y actualiza los indicadores visuales de stock.

Esquema de pruebas

Tipo de Prueba	Herramienta	Qué se va a probar	Criterios de Aceptación / Cobertura	Entornos
Unitarias	Jest	Frontend: Componentes visuales aislados y hooks. Backend: Esquemas de validación de Zod y funciones críticas (lógica de caducidades y alertas).	Cobertura de código mínima del 80% en módulos principales.	Desarrollo (Local) y CI (GitHub Actions)
Integración	Jest y Supertest	Comunicación entre controladores, servicios y base de datos (PostgreSQL/Supabase).	Operación libre de anomalías en transacciones de base de datos.	Desarrollo, CI y Staging

End-to-End (E2E)	Cypress / Playwright	Flujos completos de usuario (Registro de usuario, Login multi-tenant, Registro y edición de Lotes/Productos).	Ejecución sin fallos del 100% de los escenarios críticos en navegador.	Staging
Rendimiento	k6 (Grafana)	Capacidad de respuesta, latencia y tolerancia a fallos del backend bajo estrés.	Éxito en peticiones (checks) > 95%, fallos < 1%, latencia p(95) < 200ms.	Staging y Producción

1. Pruebas Unitarias

Su objetivo es validar el correcto funcionamiento de bloques de código individuales de manera aislada (sin interactuar con servicios externos o bases de datos).

- **En el Backend:** Se verifican los validadores de esquemas de Zod (comprobando que rechacen datos incorrectos de productos y lotes) y la lógica de negocio pura, como el cálculo de fechas de caducidad.
- **En el Frontend:** Se prueban componentes visuales aislados (botones, inputs, modales) y funciones de utilidad, garantizando que el árbol de componentes de React se renderice de forma consistente bajo diferentes estados de carga.
- **Integración Continua:** Se ejecutan localmente en desarrollo y de forma obligatoria en cada Pull Request utilizando **GitHub Actions** para evitar regresiones de código.

2. Pruebas de Integración

Comprueban la interacción fluida entre múltiples componentes del backend trabajando de manera conjunta.

- Utilizan **Supertest** para simular llamadas HTTP reales a los endpoints de la API (como **POST /products** o **PUT /lote/:id_lote**) sin levantar el servidor Express de forma física.
- Validan que la capa de servicios procese correctamente las llamadas del controlador, que se aplique la seguridad de sesión de **Better-Auth** y que las consultas/escrituras en la base de datos de PostgreSQL (Supabase) sean correctas y consistentes.

3. Pruebas de Extremo a Extremo (E2E)

Simulan el comportamiento exacto de un usuario real interactuando con la interfaz de usuario en el navegador.

- Validan flujos completos que cruzan múltiples vistas, como: iniciar sesión, navegar al inventario, dar de alta un producto, registrarle un lote, editar la cantidad y verificar que el dashboard refleje la alerta de stock correcto o de caducidad inminente.
- Se corren principalmente en entornos de **Staging** (pre-producción) sobre navegadores reales (Chrome/Firefox/Webkit) de manera headless para garantizar la calidad antes de liberar actualizaciones críticas a producción.

4. Pruebas de Rendimiento y Carga

Evalúan el comportamiento del servidor backend RESTful bajo condiciones de alta concurrencia y tráfico masivo continuo.

- Utilizan la herramienta **k6** de Grafana para simular escenarios de hasta 50 usuarios virtuales (**VUs**) realizando peticiones simultáneas sobre endpoints críticos.
- Se enfocan en asegurar que la API mantenga un porcentaje de fallos (**http_req_failed**) menor al 1% y que el tiempo medio de respuesta de servidor esté por debajo del percentil de 95ms para garantizar una navegación fluida a los usuarios concurrentes en escenarios de alta demanda comercial.